

## Tree Problem Bank

### Legend

★ = Core Placement Problem (Mandatory)

Language-Agnostic DSA Problem Bank | Build TreeNode from scratch. Covers Binary Tree, BST, Heap, AVL, Trie, and Segment Tree.

### Section 1: Binary Tree — Internals

---

- ★ Create a TreeNode (Data, Left, Right Pointers)
- ★ Build a Binary Tree (Manual / Level Order Input)
- ★ Inorder Traversal (Left → Root → Right)
- ★ Preorder Traversal (Root → Left → Right)
- ★ Postorder Traversal (Left → Right → Root)
- ★ Level Order Traversal (BFS using Queue)

Iterative Inorder Traversal using Stack

Iterative Preorder Traversal using Stack

Iterative Postorder Traversal using Two Stacks

- ★ Count Total Number of Nodes in a Binary Tree
- ★ Count Leaf Nodes

Count Internal (Non-Leaf) Nodes

### Section 2: Binary Search Tree (BST) — Internals

---

- ★ Insert a Node into a BST
- ★ Search for a Value in a BST
- ★ Delete a Node from a BST (Leaf, One Child, Two Children)
- ★ Find Minimum and Maximum in a BST
- ★ Inorder Traversal of BST gives Sorted Output (Verify)

Build a BST from a Given Array

Convert Sorted Array to Balanced BST

### Section 3: Tree Properties and Measurements

---

- ★ Height (Maximum Depth) of a Binary Tree
  - ★ Diameter of a Binary Tree
  - ★ Check if Binary Tree is Balanced
- Minimum Depth of a Binary Tree
- ★ Count Nodes at a Given Level (Level Width)
- Maximum Width of a Binary Tree
- Sum of All Node Values

### Section 4: Tree Views and Boundary

---

- ★ Left View of a Binary Tree
- ★ Right View of a Binary Tree

### ★ Top View of a Binary Tree

Bottom View of a Binary Tree

### ★ Boundary Traversal of a Binary Tree

Vertical Order Traversal of a Binary Tree

Diagonal Traversal of a Binary Tree

## Section 5: Path Problems

---

### ★ Root to Leaf Path Sum Check

### ★ Print All Root to Leaf Paths

### ★ Path with Maximum Sum

Find All Paths with Given Sum

### ★ Lowest Common Ancestor (LCA) in Binary Tree

Lowest Common Ancestor in BST

### ★ Distance between Two Nodes in a Binary Tree

Kth Ancestor of a Node

## Section 6: Tree Transformation

---

### ★ Mirror / Invert a Binary Tree

### ★ Check if Two Trees are Mirror Images

### ★ Check if Two Binary Trees are Identical

Flatten Binary Tree to Linked List (Preorder)

### ★ Convert Binary Tree to its Sum Tree

Convert BST to Greater Sum Tree

## Section 7: Special Tree Problems

---

### ★ Check if a Binary Tree is a BST

### ★ Check if a Binary Tree is Symmetric

### ★ Zigzag Level Order Traversal

Find Kth Smallest and Kth Largest Element in BST

### ★ Serialize and Deserialize a Binary Tree

Construct Binary Tree from Inorder and Preorder

Construct Binary Tree from Inorder and Postorder

## Section 8: Heap (Priority Queue) — Internals and Patterns

---

### ★ Build a Min-Heap from an Array (Heapify)

### ★ Build a Max-Heap from an Array

### ★ Insert an Element into a Heap

### ★ Delete the Root (Extract Min or Max)

### ★ Heap Sort Algorithm

Find Kth Largest Element using Min-Heap

### ★ Merge K Sorted Arrays using Min-Heap

Running Median using Two Heaps

## Section 9: AVL Tree — Internals (Good to Know)

---

Understand Balance Factor and AVL Property  
Implement Left Rotation and Right Rotation  
Insert a Node into an AVL Tree with Rebalancing  
Delete a Node from an AVL Tree with Rebalancing  
Check if a Binary Search Tree is an AVL Tree

## Section 10: Trie and Segment Tree (Advanced Practice)

---

★ **Implement a Trie — Insert and Search a Word**  
★ **Implement a Trie — Starts With (Prefix Check)**  
Implement a Trie — Delete a Word  
Count Words with a Given Prefix using Trie  
★ **Implement a Segment Tree — Build, Range Query, Point Update**  
Segment Tree — Range Minimum Query  
Segment Tree — Range Maximum Query

## Section 11: Real-World Applications of Trees

---

★ **Expression Tree — Build and Evaluate Arithmetic Expression**  
★ **File System Directory Tree — Represent Folders and Files as Tree**  
★ **Huffman Encoding using Heap — Build Optimal Prefix Code Tree**  
Auto-Complete Feature using Trie — Suggest Words by Prefix  
★ **Organisational Hierarchy Tree — Find Manager, Subordinates, and LCA**  
Decision Tree Simulation — Yes/No Branching for a Simple Classifier  
★ **Database Index using BST — Efficient Record Lookup by Key**  
Priority Task Scheduler using Heap — Execute Highest Priority Task First